



Git & GitHub Complete Commands Guide

A comprehensive reference covering 33 essential Git commands — from installation and configuration to branching, merging, and advanced history management. Master version control and collaborate with confidence.

VERSION CONTROL

33 COMMANDS

REAL-WORLD USAGE

Getting Started: Installation & Configuration

Before writing a single line of code, Git must be properly installed and configured. These foundational commands establish your identity in every commit you make.

1

Verify Installation

```
git --version
```

Confirms Git is installed on your system. Run this first before starting any project to ensure your environment is ready.

2

Set Username

```
git config --global user.name "Sindhu"
```

Configures your Git username globally. This name appears in every commit, helping teammates identify who made which changes.

3

Set Email

```
git config --global user.email "abc@gmail.com"
```

Configures your Git email globally. Associates all commits with your GitHub account for proper attribution.

4

Check Config

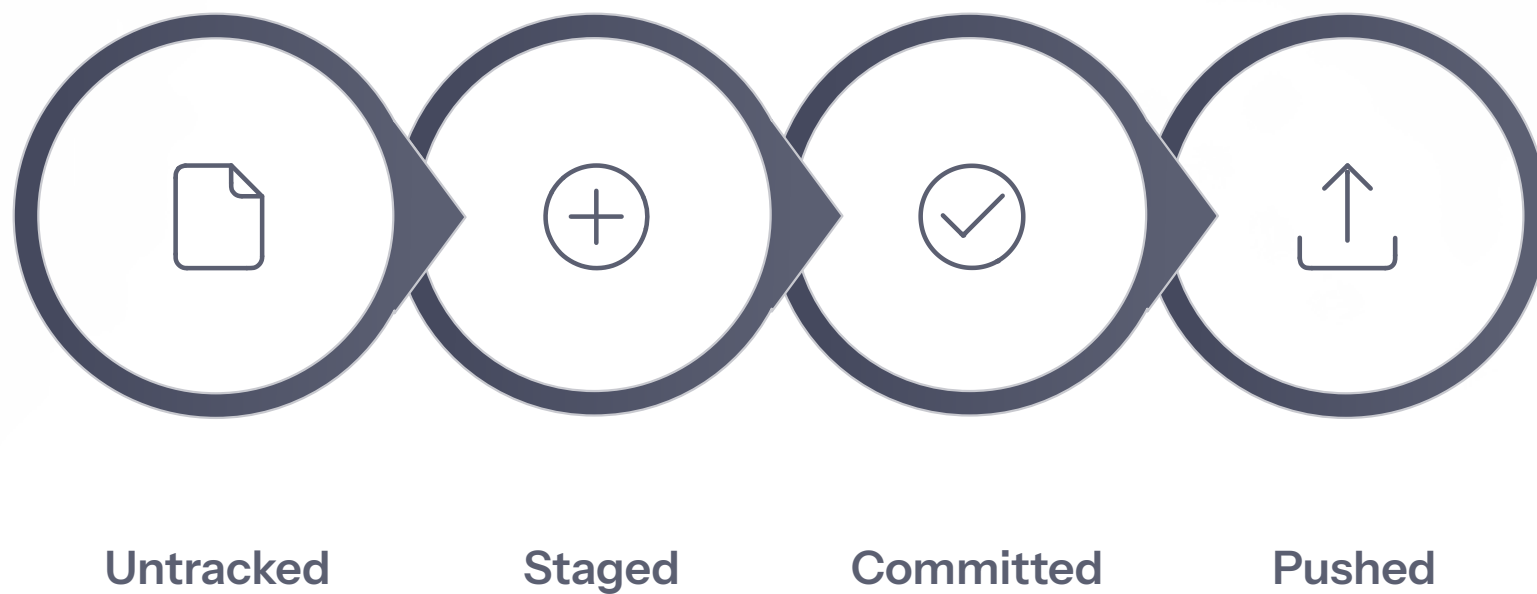
```
git config --list
```

Displays all configured Git settings. Use this to verify your username, email, and other configurations are correct.

 Always configure your username and email before making your first commit — these details are permanently embedded in your project history.

Initializing & Tracking Your Repository

Once Git is configured, the next step is creating a repository and understanding the lifecycle of your files — from untracked to committed.



Understanding this lifecycle is the foundation of effective version control. Every file in your project moves through these stages as you develop.

Initialize Repository

```
git init
```

Creates a new local Git repository inside your project folder. Git begins tracking all files and changes from this point forward.

Check Repository Status

```
git status
```

Shows modified, staged, committed, and untracked files. Run this frequently to understand the current state of your project before making any commit.

Compare Code Changes

```
git diff
```

Shows line-by-line differences between modified and committed code. Essential for reviewing exactly what changed before committing.

Staging, Committing & Viewing History

The core Git workflow revolves around three actions: staging your changes, committing them to history, and reviewing what was committed. These commands form the backbone of daily development.

1

Stage Changes

```
git add .
```

Moves all changed files to the staging area, preparing them for commit.

2

Commit Changes

```
git commit -m "Initial Commit"
```

Saves staged changes permanently into Git history with a descriptive message.

3

View History

```
git log --oneline
```

Displays commit history in a compact format for quick review of past changes.

Amend Last Commit

```
git commit --amend -m "Updated Message"
```

Updates the previous commit message or includes additional staged changes. Ideal for fixing typos or small mistakes in your most recent commit — before pushing to remote.

Blame Command

```
git blame file.txt
```

Shows who modified each line in a file, along with the commit reference. Invaluable for debugging — quickly identify code ownership and trace the history of any specific line.

Connecting to Remote Repositories

Git's true power emerges when you connect your local repository to a remote host like GitHub. These commands bridge the gap between your machine and the cloud.

Clone Repository

```
git clone  
https://github.com/user/repo  
.git
```

Downloads a remote GitHub repository to your local machine. Team members use this to get a full copy of the project with complete history.

Add Remote

```
git remote add origin  
repositoryURL
```

Connects your local repository to a GitHub repository. Required before you can push or pull code between local and remote.

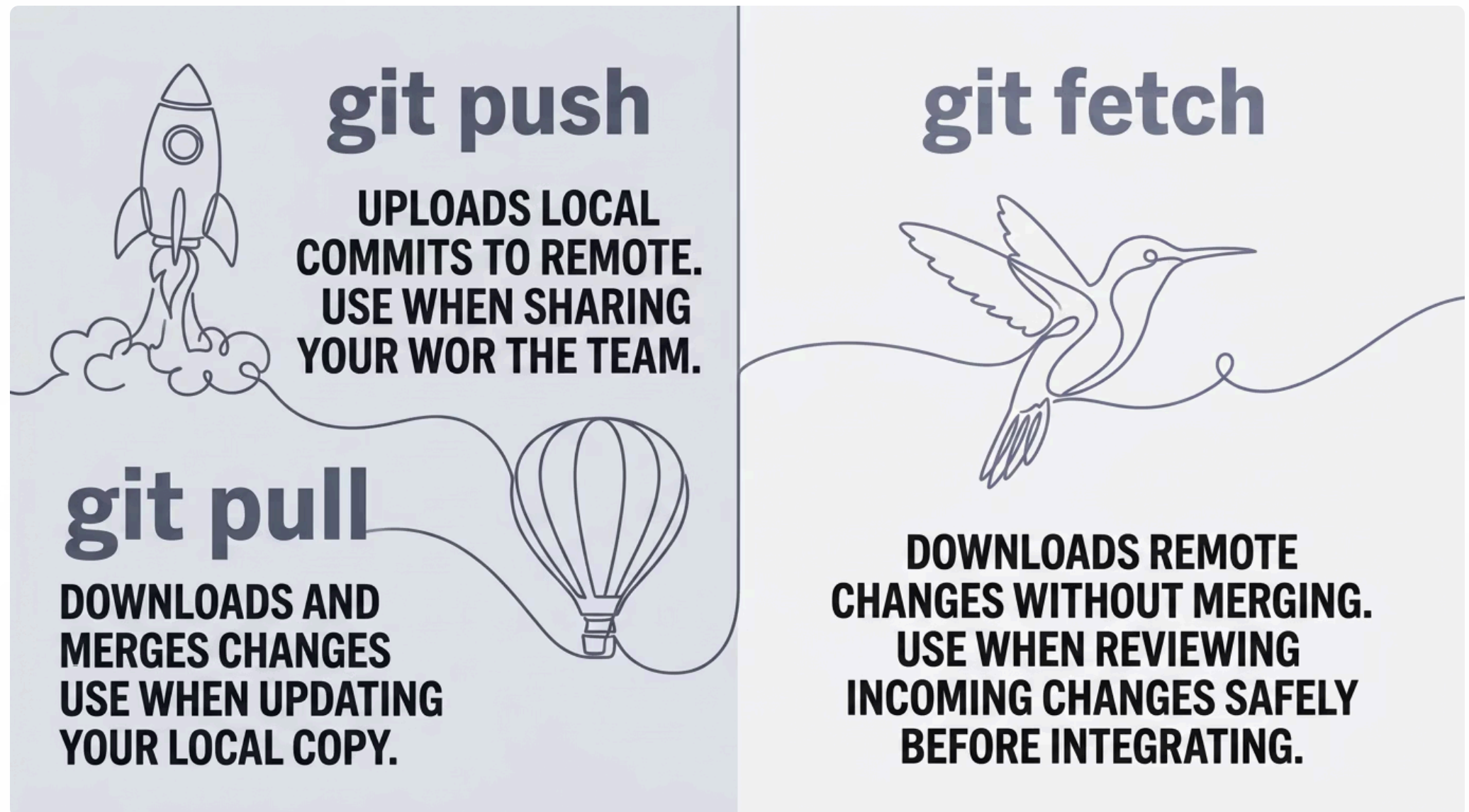
Verify Remote

```
git remote -v
```

Displays all configured remote repository URLs. Use this to confirm you're connected to the correct GitHub repository.

Syncing Code: Push, Pull & Fetch

Keeping your local and remote repositories in sync is critical for team collaboration. These three commands handle all code synchronization — each with a distinct purpose.



git push — Share Your Work

Uploads local commits to the remote GitHub repository. After committing locally, push to make your changes available to all team members. Use `git push -u origin main` for the first push to set the upstream branch.

git pull — Stay Updated

Downloads the latest code changes from remote and automatically merges them into your local branch. Run this at the start of each work session to ensure you're working with the most current codebase.

git fetch — Preview Before Merging

Downloads the latest remote changes without merging them. Lets you inspect what's coming in before integrating — a safer approach when you want to review changes first.

Branching & Merging

Branches are Git's most powerful feature — they let developers work on features, fixes, or experiments in complete isolation without affecting the main codebase. Merging brings that work back together.

01	02	03
Create & Switch Branch	Switch Between Branches	Merge Branch
<code>git checkout -b feature-login</code>	<code>git switch feature-login</code>	<code>git merge feature-login</code>
Creates a new branch and immediately switches to it. Developers use this to work independently on a feature without touching the main branch.	Moves from one branch to another. The modern, cleaner alternative to <code>git checkout</code> for switching branches. Enables working on multiple features simultaneously.	Combines the specified branch's code into the current branch. Used to integrate completed features into the main project after review and testing.
04	05	
Delete Local Branch	Delete Remote Branch	
<code>git branch -d feature-login</code>	<code>git push origin --delete feature-login</code>	
Deletes a local branch after it has been successfully merged. Keeps your repository clean and free of stale branches.	Removes the branch from the remote GitHub repository. Cleans up unused branches from the shared remote, keeping the team's workspace organized.	

Undoing Changes & Advanced History Management

Mistakes happen. Git provides multiple strategies for undoing changes — each suited to different situations. Understanding which tool to use is critical for safe recovery.

Restore File	Soft Reset	Revert Commit
<pre>git restore file.txt</pre>	<pre>git reset --soft HEAD~1</pre>	<pre>git revert commitID</pre>
Removes local modifications from a specific file, restoring it to the last committed state. Use when changes to a file are unnecessary or accidental.	Removes the last commit while keeping all code changes staged. Perfect for correcting commit mistakes without losing any work.	Creates a new commit that reverses the changes of a previous commit. The safest way to undo commits in shared repositories — history is preserved.

Temporary Save with Stash

```
git stash
```

Temporarily stores all uncommitted changes so you can switch branches without losing unfinished work. Retrieve your stash later with `git stash pop`.

When to Use Each Undo Strategy

- **restore** — Undo changes in a single file (not yet staged)
- **reset --soft** — Undo last commit, keep changes staged
- **revert** — Safely undo a commit already pushed to remote
- **stash** — Temporarily shelve work to switch context

⚠️ Never use `git reset --hard` on commits that have already been pushed to a shared remote repository — it rewrites history and can cause serious issues for teammates.

Advanced Commands: Cherry-Pick, Rebase, Tags & More

These powerful commands give you fine-grained control over your project history, release management, and repository hygiene. Used correctly, they keep your codebase clean and professional.



Cherry-Pick

```
git cherry-pick commitID
```

Copies a specific commit from another branch into the current branch. Use when you need only one particular change — not the entire branch — moved across.



Create Tag

```
git tag v1.0
```

Creates a version tag at the current commit. Tags mark important releases and milestones, making it easy to identify and return to specific versions.

Git Ignore

```
.gitignore
```

A special file that tells Git which files and folders to never track. Prevents sensitive data, build artifacts, and temporary files from being committed.

- Exclude `node_modules/` and build folders
- Protect secrets like `.env` files
- Skip OS files like `.DS_Store`
- Ignore IDE config folders like `.vscode/`



Rebase

```
git rebase main
```

Moves your branch commits on top of the updated main branch. Creates a clean, linear project history — preferred over merge in many professional workflows.



Push Tag

```
git push origin v1.0
```

Uploads the created tag to GitHub. Shares release versions with the entire team so everyone can reference the same milestone.

Clean Untracked Files

```
git clean -f
```

Permanently removes untracked files from your working directory. Use this to eliminate temporary files, generated artifacts, or leftover build outputs that clutter your project.



This action is irreversible. Run `git clean -n` first to preview which files will be deleted before executing.

Quick Reference: All 33 Commands at a Glance

A complete summary of every Git command covered in this guide, organized by category for fast lookup during your daily workflow.

Category	Command	Purpose
Setup	git --version	Verify Git installation
Setup	git config --global user.name	Set global username
Setup	git config --global user.email	Set global email
Setup	git config --list	View all Git settings
Repository	git init	Initialize local repository
Repository	git status	Check file states
Repository	git diff	Compare code changes
Staging	git add .	Stage all changes
Commits	git commit -m "msg"	Save staged changes
Commits	git log --oneline	View compact history
Commits	git commit --amend	Edit last commit
Commits	git blame file.txt	Show line-by-line authors
Remote	git clone URL	Download remote repo
Remote	git remote add origin URL	Connect to remote
Remote	git remote -v	Verify remote URLs
Sync	git push	Upload commits to remote
Sync	git pull	Download and merge changes
Sync	git fetch	Download without merging
Branching	git checkout -b branch	Create and switch branch
Branching	git switch branch	Switch to branch
Branching	git merge branch	Merge branch into current
Branching	git branch -d branch	Delete local branch
Branching	git push origin --delete branch	Delete remote branch
Undo	git restore file.txt	Discard file changes
Undo	git reset --soft HEAD~1	Undo last commit, keep changes
Undo	git revert commitID	Safely reverse a commit
Undo	git stash	Temporarily save changes
Advanced	git cherry-pick commitID	Copy specific commit
Advanced	git rebase main	Reapply commits on main
Releases	git tag v1.0	Create version tag
Releases	git push origin v1.0	Push tag to remote
Hygiene	.gitignore	Exclude files from tracking
Hygiene	git clean -f	Remove untracked files

 You've now covered all 33 essential Git commands. Practice these daily and they'll become second nature — the foundation of every professional developer's workflow.